

Packet Sniffer Homework 1 Report

pktsniffer.py — Header Summary & Packet Filtering

Name: Abdul Aleem Shaik Gadda

Course / Section: CSCI 651

Date: 25 September 2026

Pcap file(s) used: test_wifi.pcap

1. Introduction

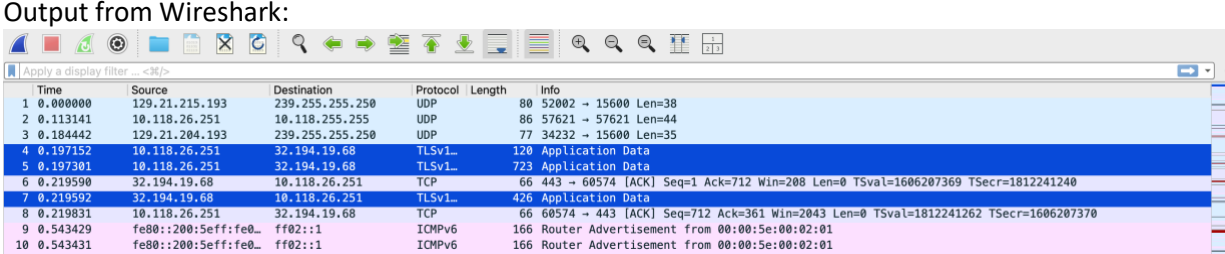
The program, `pktsniffer.py`, is a command-line packet analysis tool built on Scapy that reads packets from a .pcap file and prints their Ethernet, IP (IPv4/IPv6/ARP), and transport-layer (TCP/UDP/ICMP/ICMPv6) header fields, along with a summary, as formatted tables. It also supports a small filter expression language (port, host, net, tcp, udp, icmp, arp, combinable with and/or/not), so specific subsets of traffic can be isolated. It was tested against a sample Wi-Fi packet capture file using Wireshark, captured and run from the command line (e.g., `python3 pktsniffer.py -r test_wifi.pcap`), with additional runs adding filter expressions and the `-c/-n` flags to verify both the header-parsing and filtering logic.

2. Output Comparison: Program vs. Wireshark

The first few and last outputs from the pcap file using the `pktsniffer` script and Wireshark software are shown below.

2.1 First Few Packets

Output from Wireshark:



Output from the pktsniffer

```
(base) aleem@Abduls-Macbook-Air-3:CS1C651 % python pktsniffer.py -r test_wifi.pcap -c 10
Total packets read: 10
Only the first 10 packets were read.

Ethernet Header Information:
 packet_size_bytes destination_mac_address source_mac_address ether_type
0 80 26:80:df:ad:5f:7a 30:b6:4f:86:f6:ed IPv4
1 86 ff:ff:ff:ff:ff:ff 26:80:df:ad:5f:7a IPv4
2 77 26:80:df:ad:5f:7a 30:b6:4f:86:f6:ed IPv4
3 120 10:70:fd:02:40:e0 26:80:df:ad:5f:7a IPv4
4 723 10:70:fd:02:40:e0 26:80:df:ad:5f:7a IPv4
5 66 26:80:df:ad:5f:7a 10:70:fd:02:40:e0 IPv4
6 426 26:80:df:ad:5f:7a 10:70:fd:02:40:e0 IPv4
7 66 10:70:fd:02:40:e0 26:80:df:ad:5f:7a IPv4
8 166 26:80:df:ad:5f:7a 30:b6:4f:86:f6:2d IPv6
9 166 26:80:df:ad:5f:7a 30:b6:4f:86:f6:2d IPv6

IP Header Information:
 version header_length_bytes Type of Service total_length_bytes ... protocol header_checksum source_ip_address destination_ip_address
0 4 20 0 66 ... udp 58544 129.21.215.193 239.255.255.250
1 4 20 0 72 ... udp 19575 10.118.26.251 10.118.255.255
2 4 20 0 63 ... udp 63333 129.21.204.193 239.255.255.250
3 4 20 2 106 ... tcp 57621 10.118.26.251 32.194.19.68
4 4 20 2 709 ... tcp 57018 10.118.26.251 32.194.19.68
5 4 20 0 52 ... tcp 55775 32.194.19.68 10.118.26.251
6 4 20 2 412 ... tcp 55412 32.194.19.68 10.118.26.251
7 4 20 0 52 ... tcp 57677 10.118.26.251 32.194.19.68
8 6 40 0 152 ... ipv6_icmp <NA> fe80::200:5eff:fe00:201 ff02::1
9 6 40 0 152 ... ipv6_icmp <NA> fe80::200:5eff:fe00:201 ff02::1

[10 rows x 12 columns]

Encapsulated Packet Header Information (TCP / UDP / ICMP / ICMPv6):
 protocol source_port destination_port sequence_number acknowledgment_number header_length_bytes flags window_size checksum urgent_pointer length_bytes icmp_type icmp_code
0 udp 52002 15600 <NA> <NA> 8 None <NA> 2352 <NA> <NA> 46 <NA> <NA>
1 udp 57621 57621 <NA> <NA> 8 None <NA> 13536 <NA> <NA> 52 <NA> <NA>
2 udp 34232 15600 <NA> <NA> 8 None <NA> 46822 <NA> <NA> 43 <NA> <NA>
3 tcp 60574 443 322990663 2425605727 32 PA 2048 50899 0 <NA> <NA> <NA> <NA>
4 tcp 60574 443 322990717 2425605727 32 PA 2048 53549 0 <NA> <NA> <NA> <NA>
5 tcp 443 60574 2425605727 322991374 32 A 208 6200 0 <NA> <NA> <NA> <NA>
6 tcp 443 60574 2425605727 322991374 32 PA 208 34092 0 <NA> <NA> <NA> <NA>
7 tcp 60574 443 322991374 2425606087 32 A 2043 3982 0 <NA> <NA> <NA> <NA>
8 icmpv6 <NA> <NA> <NA> <NA> <NA> None <NA> 41065 <NA> <NA> 134 0
9 icmpv6 <NA> <NA> <NA> <NA> <NA> None <NA> 41065 <NA> <NA> 134 0
```

We can see from the figure that the `pktsniffer` provides the Ethernet Header Information for each packet, including the total packet size in bytes, the destination MAC address, the source MAC address, and the EtherType field (e.g., IPv4, IPv6, ARP). These values match what Wireshark reports in its Ethernet layer for the same packets, confirming that the program correctly parses the link-layer header.

We can see that a few packets using the IPv6 ICMP protocol have the info “Router Advertisement”. These are ICMPv6 Neighbor Discovery messages (type 134) that a router periodically multicasts to `ff02::1` (the all-nodes link-local multicast address) to announce its presence and advertise configuration information. Also, the pktsniffer provides a summary of IP and encapsulated header information, including transport-layer protocols such as “UDP” and “TCP”. However, we can see that Wireshark is more advanced at summarizing the TCP protocol as Transport Layer Security (TLS), where some application data is being encrypted and handled over the communication.

2.2 Last Few Packets

The last few packets from the output of pktsniffer and Wireshark are shown below.

Output from Wireshark:

28729	121.145130	129.21.66.243	239.255.255.250	UDP	77 49671 → 15600 Len=35
28730	121.237219	fe80::2e21:310f:b2...	ff02::1	ICMPv6	134 Router Advertisement from 2c:21:31:4e:6b:78
28731	121.272382	10.118.26.251	134.224.4.174	TCP	54 [TCP Keep-Alive] 64169 → 443 [ACK] Seq=1166 Ack=1011 Win=2048 Len=0
28732	121.277917	129.21.131.234	239.255.255.250	UDP	80 34770 → 15600 Len=38
28733	121.307007	134.224.4.174	10.118.26.251	TCP	66 [TCP Keep-Alive ACK] 443 → 64169 [ACK] Seq=1011 Ack=1167 Win=9 Len=0 TSval=1695420556 TSecr=1

Output from the pktsniffer

```
(base) aileen@Abduls-MacBook-Air-3 CS1C651 % python pktsniffer.py -r test_wifi.pcap
Total packets read: 28733
All packets were read from the pcap file.

Ethernet Header Information:
packet_size_bytes destination_mac_address source_mac_address ether_type
0 80 26:80:df:ad:5f:7a 30:b6:4f:86:f6:ed IPv4
1 86 ff:ff:ff:ff:ff:ff 26:80:df:ad:5f:7a IPv4
2 77 26:80:df:ad:5f:7a 30:b6:4f:86:f6:ed IPv4
3 120 10:70:fd:02:40:e0 26:80:df:ad:5f:7a IPv4
4 723 10:70:fd:02:40:e0 26:80:df:ad:5f:7a IPv4
...
28728 77 26:80:df:ad:5f:7a 30:b6:4f:86:f6:ed IPv4
28729 134 26:80:df:ad:5f:7a 2c:21:31:4e:6b:78 IPv6
28730 54 10:70:fd:02:40:e0 26:80:df:ad:5f:7a IPv4
28731 80 26:80:df:ad:5f:7a 30:b6:4f:86:f6:ed IPv4
28732 66 26:80:df:ad:5f:7a 10:70:fd:02:40:e0 IPv4

[28733 rows x 4 columns]

IP Header Information:
version header_length_bytes Type of Service total_length_bytes identification ... time_to_live protocol header_checksum source_ip_address destination_ip_address
0 4 20 0 66 3881 ... 62 udp 58544 129.21.215.193 239.255.255.250
1 4 20 0 72 65895 ... 64 udp 19575 10.118.26.251 10.118.255.255
2 4 20 0 63 1655 ... 63 udp 63333 129.21.284.193 239.255.255.250
3 4 20 2 186 0 ... 64 tcp 57621 10.118.26.251 32.194.19.68
4 4 20 2 789 0 ... 64 tcp 57018 10.118.26.251 32.194.19.68
...
28728 4 20 0 63 28760 ... 63 udp 13971 129.21.66.243 239.255.255.250
28729 6 40 0 120 <NA> ... 255 ipv6_icmp <NA> fe80::2e21:310f:b24e:6438 ff02::1
28730 4 20 0 40 21719 ... 64 tcp 29946 10.118.26.251 134.224.4.174
28731 4 20 0 66 4285 ... 2 udp 29428 129.21.131.234 239.255.255.250
28732 4 20 0 52 28242 ... 57 tcp 8819 134.224.4.174 10.118.26.251

[28733 rows x 12 columns]

Encapsulated Packet Header Information (TCP / UDP / ICMP / ICMPv6):
protocol source_port destination_port sequence_number acknowledgment_number header_length_bytes flags window_size checksum urgent_pointer length_bytes icmp_type icmp_code
0 udp 52802 15600 <NA> <NA> 8 None <NA> 2352 <NA> 46 <NA> <NA>
1 udp 57621 15600 <NA> <NA> 8 None <NA> 13536 <NA> 52 <NA> <NA>
2 udp 34232 15600 <NA> <NA> 8 None <NA> 46822 <NA> 43 <NA> <NA>
3 tcp 68574 443 322998063 2425685727 32 PA 2048 50899 0 <NA> <NA> <NA>
4 tcp 68574 443 3229980717 2425685727 32 PA 2048 53549 0 <NA> <NA> <NA>
...
28717 udp 49671 15600 <NA> <NA> 8 None <NA> 1126 <NA> 43 <NA> <NA>
28718 icmpv6 <NA> <NA> <NA> <NA> <NA> None <NA> 46376 <NA> <NA> 134 0
28719 tcp 64169 443 3486638853 599422218 20 A 2048 52756 0 <NA> <NA> <NA>
28720 udp 34770 15600 <NA> <NA> 8 None <NA> 41047 <NA> 46 <NA> <NA>
28721 tcp 443 64169 599422218 3486638854 32 A 9 12870 0 <NA> <NA> <NA>
```

We can see that the outputs are similar for the last few packets, there are a few packets that remain filtered by pktsniffer, such as packets using TCP, UDP, ICMP, and ICMPv6 protocols, whereas Wireshark provides the complete summary.

3. Filtering Command Demonstrations

This section shows each supported filter expression in action — comparing the unfiltered packets in Wireshark against pktsniffer's filtered output — to verify that each primitive (protocol, port, host, net, not, and, or) correctly narrows the results to only the matching packets.

3.1 Single Protocol — tcp

Shows the packets that use tcp transport protocol

Command used: `python3 pktsniffer.py -r file.pcap -c 10 tcp`

The `-c` argument is used to read the first 10 packets of the file.

Output Wireshark:

Time	Source	Destination	Protocol	Length	Info
1 0.000000	129.21.215.193	239.255.255.250	UDP	80	52002 → 15600 Len=38
2 0.113141	10.118.26.251	10.118.255.255	UDP	86	57621 → 57621 Len=44
3 0.184442	129.21.204.193	239.255.255.250	UDP	77	34232 → 15600 Len=35
4 0.197152	10.118.26.251	32.194.19.68	TLSv1..	120	Application Data
5 0.197301	10.118.26.251	32.194.19.68	TLSv1..	723	Application Data
6 0.219590	32.194.19.68	10.118.26.251	TCP	66	443 → 60574 [ACK] Seq=1 Ack=712 Win=208 Len=0 TSval=1606207369 TSecr=1812241240
7 0.219592	32.194.19.68	10.118.26.251	TLSv1..	426	Application Data
8 0.219831	10.118.26.251	32.194.19.68	TCP	66	60574 → 443 [ACK] Seq=712 Ack=361 Win=2043 Len=0 TSval=1812241262 TSecr=1606207370
9 0.543429	fe80::200:5eff:fe0...	ff02::1	ICMPv6	166	Router Advertisement from 00:00:5e:00:02:01
10 0.543431	fe80::200:5eff:fe0...	ff02::1	ICMPv6	166	Router Advertisement from 00:00:5e:00:02:01

Output pktsniffer:

```
(base) aleem@Abduls-Macbook-Air-3 CSIC651 % python pktsniffer.py -r test_wifi.pcap -c 10 tcp
Total packets read: 10
Only the first 10 packets were read.
Filter applied: tcp
Packets matching filter: 5
Ethernet Header Information:
  packet_size_bytes destination_mac_address source_mac_address ether_type
0      120      10:70:fd:02:40:e0      26:80:df:ad:5f:7a      IPv4
1      723      10:70:fd:02:40:e0      26:80:df:ad:5f:7a      IPv4
2       66      26:80:df:ad:5f:7a      10:70:fd:02:40:e0      IPv4
3       426      26:80:df:ad:5f:7a      10:70:fd:02:40:e0      IPv4
4        66      10:70:fd:02:40:e0      26:80:df:ad:5f:7a      IPv4
IP Header Information:
  version  header_length_bytes  Type of Service  total_length_bytes  identification  flags  fragment_offset  time_to_live  protocol  header_checksum  source_ip_address  destination_ip_address
0      4      20      2      186      0      DF      0      64      tcp      57621      10.118.26.251      32.194.19.6
1      4      20      2      789      0      DF      0      64      tcp      57018      10.118.26.251      32.194.19.6
2      4      20      0      52      20333      DF      0      248      tcp      55775      32.194.19.68      10.118.26.25
3      4      20      2      412      20334      DF      0      248      tcp      55412      32.194.19.68      10.118.26.25
4      4      20      0      52      0      DF      0      64      tcp      57677      10.118.26.251      32.194.19.6
Encapsulated Packet Header Information (TCP / UDP / ICMP / ICMPv6):
  protocol  source_port  destination_port  sequence_number  acknowledgment_number  header_length_bytes  flags  window_size  checksum  urgent_pointer  length_bytes  icmp_type  icmp_code
0      tcp      60574      443      322990663      2425605727      32      PA      2048      50899      0      <NA>      <NA>      <NA>
1      tcp      60574      443      322990717      2425605727      32      PA      2048      53549      0      <NA>      <NA>      <NA>
2      tcp      443      60574      2425605727      322991374      32      A      208      6200      0      <NA>      <NA>      <NA>
3      tcp      443      60574      2425605727      322991374      32      PA      208      34092      0      <NA>      <NA>      <NA>
4      tcp      60574      443      322991374      2425606087      32      A      2043      3982      0      <NA>      <NA>      <NA>
```

We can see that the pktsniffer successfully picks up the filtered packets that used tcp protocol for the transmission.

3.2 Single Protocol — UDP

Shows the packets that use the UDP transport protocol.

Command used: `python3 pktsniffer.py -r file.pcap -c 25 udp`

The `-c` argument is used to read the first 25 packets of the file.

Output Wireshark:

1	0.000000	129.21.215.193	239.255.255.250	UDP	80	52002 → 15600	Len=38
2	0.113141	10.118.26.251	10.118.255.255	UDP	86	57621 → 57621	Len=44
3	0.184442	129.21.204.193	239.255.255.250	UDP	77	34232 → 15600	Len=35
4	0.197152	10.118.26.251	32.194.19.68	TLsv1_	120	Application Data	
5	0.197301	10.118.26.251	32.194.19.68	TLsv1_	723	Application Data	
6	0.219590	32.194.19.68	10.118.26.251	TCP	66	443 → 60574 [ACK] Seq=1	Ack=712 Win=208 Len=0 TSval=1606207369 TSecr=1812241240
7	0.219592	32.194.19.68	10.118.26.251	TLsv1_	426	Application Data	
8	0.219831	10.118.26.251	32.194.19.68	TCP	66	60574 → 443 [ACK] Seq=712	Ack=361 Win=2043 Len=0 TSval=1812241262 TSecr=1606207370
9	0.543429	fe80::200:5eff:fe0...	ff02::1	ICMPv6	166	Router Advertisement from 00:00:5e:00:02:01	
10	0.543431	fe80::200:5eff:fe0...	ff02::1	ICMPv6	166	Router Advertisement from 00:00:5e:00:02:01	
11	0.680464	10.118.26.251	134.224.4.174	TLsv1_	290	Application Data	
12	0.694146	129.21.105.152	239.255.255.250	UDP	77	52838 → 15600	Len=35
13	0.720957	134.224.4.174	10.118.26.251	TCP	66	443 → 64169 [ACK] Seq=1	Ack=225 Win=9 Len=0 TSval=1695299970 TSecr=1873588980
14	0.732925	134.224.4.174	10.118.26.251	TLsv1_	268	Application Data	
15	0.733027	10.118.26.251	134.224.4.174	TCP	66	64169 → 443 [ACK] Seq=225	Ack=203 Win=2045 Len=0 TSval=1873589033 TSecr=1695299981
16	1.028472	129.21.248.189	239.255.255.250	UDP	77	48563 → 15600	Len=35
17	1.028474	10.48.4.102	239.255.255.250	UDP/X_	1008	3702 → 3702	Len=966
18	1.028476	10.48.4.102	239.255.255.250	UDP/X_	1008	3702 → 3702	Len=966
19	1.045702	10.56.6.143	239.255.255.250	UDP/X_	1008	3702 → 3702	Len=966
20	1.120909	129.21.204.116	239.255.255.250	UDP	77	37259 → 15600	Len=35
21	1.120911	129.21.66.243	239.255.255.250	UDP	77	57443 → 15600	Len=35
22	1.213674	129.21.160.101	239.255.255.250	UDP/X_	1008	3702 → 3702	Len=966
23	1.215441	129.21.160.101	239.255.255.250	UDP/X_	1008	3702 → 3702	Len=966
24	1.402911	129.21.139.111	239.255.255.250	SSDP	456	NOTIFY * HTTP/1.1	

Output pktsniffer:

```
(base) aleem@Abduls-Macbook-Air-3 CS1C651 % python pktsniffer.py -r test_wifi.pcap -c 25 udp
Only the first 25 packets were read.

Filter applied: udp
Packets matching filter: 13

Ethernet Header Information:

 packet_size_bytes destination_mac_address source_mac_address ether_type
0      86      26:80:df:ad:5f:7a      30:b6:4f:86:f6:ed      IPv4
1      86      ff:ff:ff:ff:ff:ff      26:80:df:ad:5f:7a      IPv4
2      77      26:80:df:ad:5f:7a      30:b6:4f:86:f6:ed      IPv4
3      77      26:80:df:ad:5f:7a      30:b6:4f:86:f6:ed      IPv4
4      77      26:80:df:ad:5f:7a      30:b6:4f:86:f6:ed      IPv4
5     1008      26:80:df:ad:5f:7a      30:b6:4f:86:fe:2d      IPv4
6     1008      26:80:df:ad:5f:7a      30:b6:4f:86:fe:2d      IPv4
7     1008      26:80:df:ad:5f:7a      30:b6:4f:86:fe:2d      IPv4
8      77      26:80:df:ad:5f:7a      30:b6:4f:86:f6:ed      IPv4
9      77      26:80:df:ad:5f:7a      30:b6:4f:86:f6:ed      IPv4
10     1008      26:80:df:ad:5f:7a      30:b6:4f:86:fe:2d      IPv4
11     1008      26:80:df:ad:5f:7a      30:b6:4f:86:fe:2d      IPv4
12      456      26:80:df:ad:5f:7a      30:b6:4f:86:f6:ed      IPv4

IP Header Information:

 version header_length_bytes Type of Service total_length_bytes identification flags fragment_offset time_to_live protocol header_checksum source_ip_address destination_ip_address
0      4      20      0      66      3881      DF      0      62      udp      58544      129.21.215.193      239.255.255.250
1      4      20      0      72      65095      DF      0      64      udp      19575      10.118.26.251      10.118.255.255
2      4      20      0      63      1655      DF      0      63      udp      63333      129.21.204.193      239.255.255.250
3      4      20      0      63      50055      DF      0      63      udp      40318      129.21.105.152      239.255.255.250
4      4      20      0      63      9638      DF      0      63      udp      44090      129.21.248.189      239.255.255.250
5      4      20      0      994      51306      DF      0      3      udp      44304      10.48.4.102      239.255.255.250
6      4      20      0      994      51306      DF      0      3      udp      44304      10.48.4.102      239.255.255.250
7      4      20      0      994      11877      DF      0      3      udp      17637      10.56.6.143      239.255.255.250
8      4      20      0      63      52010      DF      0      63      udp      13055      129.21.204.116      239.255.255.250
9      4      20      0      63      7826      DF      0      63      udp      26905      129.21.66.243      239.255.255.250
10     4      20      0      994      64135      DF      0      3      udp      26638      129.21.160.101      239.255.255.250
11     4      20      0      994      64135      DF      0      4      udp      26382      129.21.160.101      239.255.255.250
12     4      20      0      442      1406      254      0      254      udp      47413      129.21.139.111      239.255.255.250

Encapsulated Packet Header Information (TCP / UDP / ICMP / ICMPv6):

 protocol source_port destination_port sequence_number acknowledgment_number header_length_bytes flags window_size checksum urgent_pointer length_bytes icmp_type icmp_code
0      udp      52002      15600      <NA>      <NA>      8      None      <NA>      2352      <NA>      46      <NA>      <NA>
1      udp      57621      57621      <NA>      <NA>      8      None      <NA>      13536      <NA>      52      <NA>      <NA>
2      udp      34232      15600      <NA>      <NA>      8      None      <NA>      46822      <NA>      43      <NA>      <NA>
3      udp      52838      15600      <NA>      <NA>      8      None      <NA>      53601      <NA>      43      <NA>      <NA>
4      udp      48563      15600      <NA>      <NA>      8      None      <NA>      21231      <NA>      43      <NA>      <NA>
5      udp      3702      3702      <NA>      <NA>      8      None      <NA>      9759      <NA>      974      <NA>      <NA>
6      udp      3702      3702      <NA>      <NA>      8      None      <NA>      9759      <NA>      974      <NA>      <NA>
7      udp      3702      3702      <NA>      <NA>      8      None      <NA>      31196      <NA>      974      <NA>      <NA>
8      udp      37259      15600      <NA>      <NA>      8      None      <NA>      43872      <NA>      43      <NA>      <NA>
9      udp      57443      15600      <NA>      <NA>      8      None      <NA>      58889      <NA>      43      <NA>      <NA>
10     udp      3702      3702      <NA>      <NA>      8      None      <NA>      15950      <NA>      974      <NA>      <NA>
11     udp      3702      3702      <NA>      <NA>      8      None      <NA>      15950      <NA>      974      <NA>      <NA>
12     udp      1900      1990      <NA>      <NA>      8      None      <NA>      45843      <NA>      422      <NA>      <NA>
```

We can see that the pktsniffer successfully picks up the packets using udp as the transfer protocol.

3.3 Single Protocol — icmp (ICMP + ICMPv6)

Shows the packets that use the ICMP transport protocol.

Command used: `python3 pktsniffer.py -r file.pcap -c 10 icmp`

The -c argument is used to read the first 10 packets of the file.

Output Wireshark:

Time	Source	Destination	Protocol	Length	Info
1 0.000000	129.21.215.193	239.255.255.250	UDP	80	52002 → 15600 Len=38
2 0.113141	10.118.26.251	10.118.255.255	UDP	86	57621 → 57621 Len=44
3 0.184442	129.21.204.193	239.255.255.250	UDP	77	34232 → 15600 Len=35
4 0.197152	10.118.26.251	32.194.19.68	TLSv1_	120	Application Data
5 0.197301	10.118.26.251	32.194.19.68	TLSv1_	723	Application Data
6 0.219590	32.194.19.68	10.118.26.251	TCP	66	443 → 60574 [ACK] Seq=1 Ack=712 Win=208 Len=0 TSval=1606207369 TSecr=1812241240
7 0.219592	32.194.19.68	10.118.26.251	TLSv1_	426	Application Data
8 0.219831	10.118.26.251	32.194.19.68	TCP	66	60574 → 443 [ACK] Seq=712 Ack=361 Win=2043 Len=0 TSval=1812241262 TSecr=1606207370
9 0.543429	fe80::200:5eff:fe0... ff02::1		ICMPv6	166	Router Advertisement from 00:00:5e:00:02:01
10 0.543431	fe80::200:5eff:fe0... ff02::1		ICMPv6	166	Router Advertisement from 00:00:5e:00:02:01

Output pktsniffer:

```
(base) aleem@Abduls-Macbook-Air-3 CSIC651 % python pktsniffer.py -r test_wifi.pcap -c 25 icmp
Total packets read: 25
Only the first 25 packets were read.
Filter applied: icmp
Packets matching filter: 2
Ethernet Header Information:
  packet_size_bytes destination_mac_address source_mac_address ether_type
0      166      26:80:df:ad:5f:7a    30:b6:4f:86:fe:2d    IPv6
1      166      26:80:df:ad:5f:7a    30:b6:4f:86:fe:2d    IPv6
IP Header Information:
  version  header_length_bytes  Type of Service  total_length_bytes  identification  ... time_to_live  protocol  header_checksum  source_ip_address  destination_ip_address
0      6      40      0      152      <NA>      ...      255  ipv6_icmp      <NA>  fe80::200:5eff:fe00:201  ff02::1
1      6      40      0      152      <NA>      ...      255  ipv6_icmp      <NA>  fe80::200:5eff:fe00:201  ff02::1
[2 rows x 12 columns]
Encapsulated Packet Header Information (TCP / UDP / ICMP / ICMPv6):
  protocol  source_port  destination_port  sequence_number  acknowledgment_number  header_length_bytes  flags  window_size  checksum  urgent_pointer  length_bytes  icmp_type  icmp_code
0  icmpv6      <NA>      <NA>      <NA>      <NA>      <NA>  None      <NA>      41065      <NA>      <NA>      134      0
1  icmpv6      <NA>      <NA>      <NA>      <NA>      <NA>  None      <NA>      41065      <NA>      <NA>      134      0
```

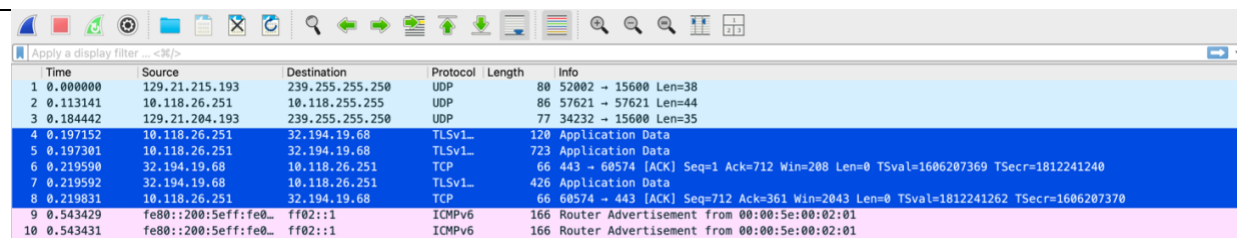
We can see that the pktsniffer successfully picks up the packets using icmp in the IP layer.

3.4 port <n>

Shows the packets that are sourced or destined to specific ports.

Command used: `python3 pktsniffer.py -r file.pcap -c 10 port 60574`

Output Wireshark:



Time	Source	Destination	Protocol	Length	Info
1 0.000000	129.21.215.193	239.255.255.250	UDP	80	52002 → 15600 Len=38
2 0.113141	10.118.26.251	10.118.255.255	UDP	86	57621 → 57621 Len=44
3 0.184442	129.21.204.193	239.255.255.250	UDP	77	34232 → 15600 Len=35
4 0.197152	10.118.26.251	32.194.19.68	TLSv1_	120	Application Data
5 0.197301	10.118.26.251	32.194.19.68	TLSv1_	723	Application Data
6 0.219590	32.194.19.68	10.118.26.251	TCP	66	443 → 60574 [ACK] Seq=1 Ack=712 Win=208 Len=0 TSval=1606207369 TSecr=1812241240
7 0.219592	32.194.19.68	10.118.26.251	TLSv1_	426	Application Data
8 0.219831	10.118.26.251	32.194.19.68	TCP	66	60574 → 443 [ACK] Seq=712 Ack=361 Win=2043 Len=0 TSval=1812241262 TSecr=1606207370
9 0.543429	fe80::200:5eff:fe0... ff02::1		ICMPv6	166	Router Advertisement from 00:00:5e:00:02:01
10 0.543431	fe80::200:5eff:fe0... ff02::1		ICMPv6	166	Router Advertisement from 00:00:5e:00:02:01

Output pktsniffer:

```
• (base) aleem@Abduls-Macbook-Air-3 CSIC651 % python pktsniffer.py -r test_wifi.pcap -c 10 port 60574

Total packets read: 10

Only the first 10 packets were read.

Filter applied: port 60574
Packets matching filter: 5

Ethernet Header Information:

packet_size_bytes destination_mac_address source_mac_address ether_type
0 120 10:70:fd:02:40:e0 26:80:df:ad:5f:7a IPv4
1 723 10:70:fd:02:40:e0 26:80:df:ad:5f:7a IPv4
2 66 26:80:df:ad:5f:7a 10:70:fd:02:40:e0 IPv4
3 426 26:80:df:ad:5f:7a 10:70:fd:02:40:e0 IPv4
4 66 10:70:fd:02:40:e0 26:80:df:ad:5f:7a IPv4

IP Header Information:

version header_length_bytes Type of Service total_length_bytes identification flags fragment_offset time_to_live protocol header_checksum source_ip_address destination_ip_address
0 4 20 2 186 0 DF 0 64 tcp 57621 10.118.26.251 32.194.19.68
1 4 20 2 709 0 DF 0 64 tcp 57018 10.118.26.251 32.194.19.68
2 4 20 2 52 20333 DF 0 248 tcp 55775 32.194.19.68 10.118.26.251
3 4 20 2 412 20334 DF 0 248 tcp 55412 32.194.19.68 10.118.26.251
4 4 20 0 52 0 DF 0 64 tcp 57677 10.118.26.251 32.194.19.68

Encapsulated Packet Header Information (TCP / UDP / ICMP / ICMPv6):

protocol source_port destination_port sequence_number acknowledgment_number header_length_bytes flags window_size checksum urgent_pointer length_bytes icmp_type icmp_code
0 tcp 60574 443 322990663 2425685727 32 PA 2048 50899 0 <NA> <NA> <NA>
1 tcp 60574 443 322990717 2425685727 32 PA 2048 53549 0 <NA> <NA> <NA>
2 tcp 443 60574 2425685727 322991374 32 A 208 6200 0 <NA> <NA> <NA>
3 tcp 443 60574 2425685727 322991374 32 PA 208 34092 0 <NA> <NA> <NA>
4 tcp 60574 443 322991374 2425686087 32 A 2043 3982 0 <NA> <NA> <NA>
```

We can see here that the pkt sniffer successfully picks the packets that are sourcing from or destined towards the port 60574.

3.5 host <addr>

Shows the packets that are sourced or destined to specific host.

Command used: python3 pktsniffer.py -r file.pcap host 10.56.5.32

Output Wireshark:

31	1.594799	52.206.163.226	10.118.26.251	TLSv1..	122	Application Data
32	1.595047	10.118.26.251	52.206.163.226	TCP	66	60578 → 443 [ACK] Seq=1059 Ack=1505 Win=2025 Len=0 TSval=3818911374 TSecr=3426917959
33	1.596117	52.206.163.226	10.118.26.251	TLSv1..	97	Application Data
34	1.596190	10.118.26.251	52.206.163.226	TCP	66	60578 → 443 [ACK] Seq=1059 Ack=1536 Win=2048 Len=0 TSval=3818911375 TSecr=3426917960
35	1.632742	10.56.5.32	239.255.255.250	UDP/X..	1008	3702 → 3702 Len=966
36	1.932203	129.21.204.120	239.255.255.250	UDP	77	46772 → 15600 Len=35
37	1.932206	fe80::32b6:4f0f:b2...	ff02::1	ICMPv6	166	Router Advertisement from 30:b6:4f:86:fe:2d
38	1.932207	fe80::32b6:4f0f:b2...	ff02::1	ICMPv6	166	Router Advertisement from 30:b6:4f:86:fe:2d
39	1.932208	10.56.5.32	239.255.255.250	UDP/X..	1008	3702 → 3702 Len=966

Output pktsniffer:

```
• (base) aleem@Abduls-Macbook-Air-3 CSIC651 % python pktsniffer.py -r test_wifi.pcap host 10.56.5.32

Total packets read: 28733
All packets were read from the pcap file.

Filter applied: host 10.56.5.32
Packets matching filter: 1

Ethernet Header Information:

packet_size_bytes destination_mac_address source_mac_address ether_type
0 1008 26:80:df:ad:5f:7a 30:b6:4f:86:fe:2d IPv4

IP Header Information:

version header_length_bytes Type of Service total_length_bytes identification flags fragment_offset time_to_live protocol header_checksum source_ip_address destination_ip_address
0 4 20 0 994 5692 DF 0 3 udp 24189 10.56.5.32 239.255.255.250

Encapsulated Packet Header Information (TCP / UDP / ICMP / ICMPv6):

protocol source_port destination_port sequence_number acknowledgment_number header_length_bytes flags window_size checksum urgent_pointer length_bytes icmp_type icmp_code
0 udp 3702 3702 <NA> <NA> 8 None <NA> 44156 <NA> 974 <NA> <NA>
```

We can see here that the pkt sniffer successfully picks the packets that are sourcing from or destined towards the host 10.56.5.32.

3.6 net <addr>[/prefix]

Shows the packets whose source or destination falls inside that subnet remain (default /24 if no prefix given).

Command used: python3 pktsniffer.py -r file.pcap net 10.56.5.32/24

Output Wireshark:

18	1.028476	10.48.4.102	239.255.255.250	UDP/X..	1008	3702 → 3702	Len=966
19	1.045702	10.56.6.143	239.255.255.250	UDP/X..	1008	3702 → 3702	Len=966
20	1.120909	129.21.204.116	239.255.255.250	UDP	77	37259 → 15600	Len=35
21	1.120911	129.21.66.243	239.255.255.250	UDP	77	57443 → 15600	Len=35
22	1.213674	129.21.160.101	239.255.255.250	UDP/X..	1008	3702 → 3702	Len=966
23	1.215441	129.21.160.101	239.255.255.250	UDP/X..	1008	3702 → 3702	Len=966
24	1.402911	129.21.139.111	239.255.255.250	SSDP	456	NOTIFY * HTTP/1.1	
25	1.469412	10.118.26.251	52.206.163.226	TLSv1..	932	Application Data	
26	1.469611	10.118.26.251	52.206.163.226	TLSv1..	258	Application Data	
27	1.573346	52.206.163.226	10.118.26.251	TCP	66	443 → 60578 [ACK] Seq=1 Ack=1059 Win=410 Len=0 TSval=3426917934 TSecr=3818911248	
28	1.583705	129.21.52.109	239.255.255.250	UDP	77	40969 → 15600	Len=35
29	1.589172	10.115.41.215	239.255.255.250	UDP	318	39817 → 1902	Len=276
30	1.594797	52.206.163.226	10.118.26.251	TCP	1514	443 → 60578 [ACK] Seq=1 Ack=1059 Win=410 Len=1448 TSval=3426917959 TSecr=3818911248 [TCP P	
31	1.594799	52.206.163.226	10.118.26.251	TLSv1..	122	Application Data	
32	1.595047	10.118.26.251	52.206.163.226	TCP	66	60578 → 443 [ACK] Seq=1059 Ack=1505 Win=2025 Len=0 TSval=3818911374 TSecr=3426917959	
33	1.596117	52.206.163.226	10.118.26.251	TLSv1..	97	Application Data	
34	1.596190	10.118.26.251	52.206.163.226	TCP	66	60578 → 443 [ACK] Seq=1059 Ack=1536 Win=2048 Len=0 TSval=3818911375 TSecr=3426917960	
35	1.632742	10.56.5.32	239.255.255.250	UDP/X..	1008	3702 → 3702	Len=966
36	1.932203	129.21.204.120	239.255.255.250	UDP	77	46772 → 15600	Len=35
37	1.932206	fe80::32b6:4f0f:b2..	ff02::1	ICMPv6	166	Router Advertisement from 30:b6:4f:86:fe:2d	
38	1.932207	fe80::32b6:4f0f:b2..	ff02::1	ICMPv6	166	Router Advertisement from 30:b6:4f:86:fe:2d	
39	1.932209	10.56.6.58	239.255.255.250	UDP/X..	1008	3702 → 3702	Len=966
40	2.078902	10.48.4.62	239.255.255.250	UDP/X..	1008	3702 → 3702	Len=966
41	2.258229	129.21.115.126	239.255.255.250	SSDP	452	NOTIFY * HTTP/1.1	

Output pktsniffer:

```
● (base) aleem@Abduls-Macbook-Air-3 CSIC651 % python pktsniffer.py -r test_wifi.pcap net 10.56.6.58/24

Total packets read: 28733
All packets were read from the pcap file.

Filter applied: net 10.56.6.58/24
Packets matching filter: 5

Ethernet Header Information:

 packet_size_bytes destination_mac_address source_mac_address ether_type
0      1008      26:80:df:ad:5f:7a      38:b6:4f:86:fe:2d      IPv4
1      1008      26:80:df:ad:5f:7a      38:b6:4f:86:fe:2d      IPv4
2      1008      26:80:df:ad:5f:7a      38:b6:4f:86:fe:2d      IPv4
3      1008      26:80:df:ad:5f:7a      38:b6:4f:86:fe:2d      IPv4
4      1008      26:80:df:ad:5f:7a      38:b6:4f:86:fe:2d      IPv4

IP Header Information:

 version  header_length_bytes  Type of Service  total_length_bytes  identification  flags  fragment_offset  time_to_live  protocol  header_checksum  source_ip_address  destination_ip_address
0      4      20      0      994      11877      DF      0      3      udp      17637      10.56.6.143      239.255.255.250
1      4      20      0      994      7116      DF      0      3      udp      22483      10.56.6.58      239.255.255.250
2      4      20      0      994      29161      DF      0      3      udp      382      10.56.6.114      239.255.255.250
3      4      20      0      994      50385      DF      0      3      udp      44602      10.56.6.205      239.255.255.250
4      4      20      0      994      23843      DF      0      3      udp      5585      10.56.6.229      239.255.255.250

Encapsulated Packet Header Information (TCP / UDP / ICMP / ICMPv6):

 protocol  source_port  destination_port  sequence_number  acknowledgment_number  header_length_bytes  flags  window_size  checksum  urgent_pointer  length_bytes  icmp_type  icmp_code
0      udp      3702      3702      <NA>      <NA>      8      None      <NA>      31196      <NA>      974      <NA>      <NA>
1      udp      3702      3702      <NA>      <NA>      8      None      <NA>      8099      <NA>      974      <NA>      <NA>
2      udp      3702      3702      <NA>      <NA>      8      None      <NA>      12915      <NA>      974      <NA>      <NA>
3      udp      3702      3702      <NA>      <NA>      8      None      <NA>      36484      <NA>      974      <NA>      <NA>
4      udp      3702      3702      <NA>      <NA>      8      None      <NA>      45823      <NA>      974      <NA>      <NA>
```

We can see here that the pkt sniffer successfully picks the packets that fall inside the subnet domain 10.56.6.32/24.

3.7 ip filter

Show the packets with IP header. Corresponding to IP version 4. IPv6 is covered under ICMP.

Command used: python3 pktsniffer.py -r file.pcap -c 10 ip

Output Wireshark:

	Time	Source	Destination	Protocol	Length	Info
1	0.000000	129.21.215.193	239.255.255.250	UDP	80	52002 → 15600 Len=38
2	0.113141	10.118.26.251	10.118.255.255	UDP	86	57621 → 57621 Len=44
3	0.184442	129.21.204.193	239.255.255.250	UDP	77	34232 → 15600 Len=35
4	0.197152	10.118.26.251	32.194.19.68	TLSv1..	120	Application Data
5	0.197301	10.118.26.251	32.194.19.68	TLSv1..	723	Application Data
6	0.219590	32.194.19.68	10.118.26.251	TCP	66	443 → 60574 [ACK] Seq=1 Ack=712 Win=208 Len=0 TSval=1606207369 TSecr=1812241240
7	0.219592	32.194.19.68	10.118.26.251	TLSv1..	426	Application Data
8	0.219831	10.118.26.251	32.194.19.68	TCP	66	60574 → 443 [ACK] Seq=712 Ack=361 Wins=2043 Len=0 TSval=1812241262 TSecr=1606207370
9	0.543429	fe80::200:5eff:fe0...	ff02::1	ICMPv6	166	Router Advertisement from 00:00:5e:00:02:01
10	0.543431	fe80::200:5eff:fe0...	ff02::1	ICMPv6	166	Router Advertisement from 00:00:5e:00:02:01

Output pktsniffer:

```
(base) aleem@Abduls-Macbook-Air-3 CSIC651 % python pktsniffer.py -r test_wifi.pcap -c 10 ip
Total packets read: 10
Only the first 10 packets were read.
Filter applied: ip
Packets matching filter: 8
Ethernet Header Information:
packet_size_bytes destination_mac_address source_mac_address ether_type
0 80 26:80:df:ad:5f:7a 30:b6:4f:86:f6:ed IPv4
1 86 ffff:ff:ff:ff:ff:ff 26:80:df:ad:5f:7a IPv4
2 77 26:80:df:ad:5f:7a 30:b6:4f:86:f6:ed IPv4
3 120 10:70:fd:02:40:e0 26:80:df:ad:5f:7a IPv4
4 723 10:70:fd:02:40:e0 26:80:df:ad:5f:7a IPv4
5 66 26:80:df:ad:5f:7a 10:70:fd:02:40:e0 IPv4
6 426 26:80:df:ad:5f:7a 10:70:fd:02:40:e0 IPv4
7 66 10:70:fd:02:40:e0 26:80:df:ad:5f:7a IPv4
IP Header Information:
version header_length_bytes Type of Service total_length_bytes ... protocol header_checksum source_ip_address destination_ip_address
0 4 20 0 66 ... udp 58544 129.21.215.193 239.255.255.250
1 4 20 0 72 ... udp 19575 10.118.26.251 10.118.255.255
2 4 20 0 63 ... udp 63333 129.21.284.193 239.255.255.250
3 4 20 2 106 ... tcp 57621 10.118.26.251 32.194.19.68
4 4 20 2 709 ... tcp 57018 10.118.26.251 32.194.19.68
5 4 20 0 52 ... tcp 55775 32.194.19.68 10.118.26.251
6 4 20 2 412 ... tcp 55412 32.194.19.68 10.118.26.251
7 4 20 0 52 ... tcp 57677 10.118.26.251 32.194.19.68
[8 rows x 12 columns]
Encapsulated Packet Header Information (TCP / UDP / ICMP / ICMPv6):
protocol source_port destination_port sequence_number acknowledgment_number header_length_bytes flags window_size checksum urgent_pointer length_bytes icmp_type icmp_code
0 udp 52002 15600 <NA> <NA> 8 None <NA> 2352 <NA> 46 <NA> <NA>
1 udp 57621 15600 <NA> <NA> 8 None <NA> 13536 <NA> 52 <NA> <NA>
2 udp 34232 15600 <NA> <NA> 8 None <NA> 46822 <NA> 43 <NA> <NA>
3 tcp 68574 443 322990663 2425605727 32 PA 2048 58899 0 <NA> <NA> <NA>
4 tcp 68574 443 322990717 2425605727 32 PA 2048 53549 0 <NA> <NA> <NA>
5 tcp 443 68574 2425605727 322991374 32 A 208 6200 0 <NA> <NA> <NA>
6 tcp 443 68574 2425605727 322991374 32 PA 208 34092 0 <NA> <NA> <NA>
7 tcp 68574 443 322991374 2425606087 32 A 2043 3982 0 <NA> <NA> <NA>
```

We can see here that the pkt sniffer successfully picks the packets that with the IPv4 Headers.

3.8 <clause> and <clause>, <clause> or <clause>

Show the packets satisfying both the clauses for “and” and either of them for “or”

Using “and” clause: python3 pktsniffer.py -r file.pcap host 10.56.5.8 and port 3702

Output Wireshark:

No.	Time	Source	Destination	Protocol	Length	Info
45	2.624233	10.56.4.127	239.255.255.250	UDP/X...	1008	3702 → 3702 Len=966
46	2.767459	10.56.10.116	239.255.255.250	UDP/X...	1008	3702 → 3702 Len=966
161	8.383543	10.56.4.158	239.255.255.250	UDP/X...	1008	3702 → 3702 Len=966
165	8.440254	10.56.5.8	239.255.255.250	UDP/X...	1008	3702 → 3702 Len=966
188	10.284600	10.56.4.31	239.255.255.250	UDP/X...	1008	3702 → 3702 Len=966
257	11.208830	10.56.4.238	239.255.255.250	UDP/X...	1008	3702 → 3702 Len=966
268	12.331438	10.56.5.22	239.255.255.250	UDP/X...	1008	3702 → 3702 Len=966
309	13.007467	10.56.5.191	239.255.255.250	UDP/X...	1008	3702 → 3702 Len=966
5671	37.205843	10.32.5.152	239.255.255.250	UDP/X...	1008	3702 → 3702 Len=966

Output pktsniffer:

```
(base) aleem@Abduls-Macbook-Air-3 CSIC651 % python pktsniffer.py -r test_wifi.pcap host 10.56.5.8 and port 3702
Total packets read: 28733
All packets were read from the pcap file.
Filter applied: host 10.56.5.8 and port 3702
Packets matching filter: 1
Ethernet Header Information:
packet_size_bytes destination_mac_address source_mac_address ether_type
0 1008 26:80:df:ad:5f:7a 30:b6:4f:86:f6:ed IPv4
IP Header Information:
version header_length_bytes Type of Service total_length_bytes identification flags fragment_offset time_to_live protocol header_checksum source_ip_address destination_ip_address
0 4 20 0 994 50457 DF 0 3 udp 44983 10.56.5.8 239.255.255.250
Encapsulated Packet Header Information (TCP / UDP / ICMP / ICMPv6):
protocol source_port destination_port sequence_number acknowledgment_number header_length_bytes flags window_size checksum urgent_pointer length_bytes icmp_type icmp_code
0 udp 3702 3702 <NA> <NA> 8 None <NA> 52553 <NA> 974 <NA> <NA>
```

We can see that the pktsniffer successfully locates the host on the specific port.

Using “or” clause: python3 pktsniffer.py -r file.pcap host 10.48.4.102 or port 3702

Output Wireshark:

No.	Time	Source	Destination	Protocol	Length	Info
17	1.028474	10.48.4.102	239.255.255.250	UDP/X..	1008	3702 → 3702 Len=966
18	1.028476	10.48.4.102	239.255.255.250	UDP/X..	1008	3702 → 3702 Len=966
19	1.045702	10.56.6.143	239.255.255.250	UDP/X..	1008	3702 → 3702 Len=966
22	1.213674	129.21.160.101	239.255.255.250	UDP/X..	1008	3702 → 3702 Len=966
23	1.215441	129.21.160.101	239.255.255.250	UDP/X..	1008	3702 → 3702 Len=966
35	1.632742	10.56.5.32	239.255.255.250	UDP/X..	1008	3702 → 3702 Len=966
39	1.932209	10.56.6.58	239.255.255.250	UDP/X..	1008	3702 → 3702 Len=966
40	2.078902	10.48.4.62	239.255.255.250	UDP/X..	1008	3702 → 3702 Len=966
42	2.379655	10.56.7.155	239.255.255.250	UDP/X..	1008	3702 → 3702 Len=966
45	2.624233	10.56.4.127	239.255.255.250	UDP/X..	1008	3702 → 3702 Len=966
46	2.767459	10.56.10.116	239.255.255.250	UDP/X..	1008	3702 → 3702 Len=966
161	8.383543	10.56.4.158	239.255.255.250	UDP/X..	1008	3702 → 3702 Len=966
165	8.440254	10.56.5.8	239.255.255.250	UDP/X..	1008	3702 → 3702 Len=966
188	10.284680	10.56.4.31	239.255.255.250	UDP/X..	1008	3702 → 3702 Len=966
257	11.208830	10.56.4.238	239.255.255.250	UDP/X..	1008	3702 → 3702 Len=966
268	12.331438	10.56.5.22	239.255.255.250	UDP/X..	1008	3702 → 3702 Len=966
309	13.087467	10.56.5.191	239.255.255.250	UDP/X..	1008	3702 → 3702 Len=966
5671	37.205843	10.32.5.152	239.255.255.250	UDP/X..	1008	3702 → 3702 Len=966
7622	49.571357	10.48.101.224	239.255.255.250	UDP/X..	1008	3702 → 3702 Len=966
12569	51.784537	10.56.4.192	239.255.255.250	UDP/X..	1008	3702 → 3702 Len=966
13061	52.131987	10.56.4.94	239.255.255.250	UDP/X..	1008	3702 → 3702 Len=966
13383	52.455770	10.32.101.243	239.255.255.250	UDP/X..	1008	3702 → 3702 Len=966
19418	55.557058	10.56.6.114	239.255.255.250	UDP/X..	1008	3702 → 3702 Len=966
19436	57.126543	10.56.5.93	239.255.255.250	UDP/X..	1008	3702 → 3702 Len=966
19453	57.828024	129.21.202.206	239.255.255.250	UDP/X..	1008	3702 → 3702 Len=966
19454	57.961748	10.56.6.205	239.255.255.250	UDP/X..	1008	3702 → 3702 Len=966
19460	58.829731	10.56.4.193	239.255.255.250	UDP/X..	1008	3702 → 3702 Len=966
19481	60.978169	10.16.233.34	239.255.255.250	UDP/X..	1008	3702 → 3702 Len=966
19503	63.036276	129.21.160.3	239.255.255.250	UDP/X..	1008	3702 → 3702 Len=966
19522	63.551974	10.56.4.196	239.255.255.250	UDP/X..	1008	3702 → 3702 Len=966
19608	69.819238	10.56.7.198	239.255.255.250	UDP/X..	1008	3702 → 3702 Len=966
19622	71.283676	10.56.5.223	239.255.255.250	UDP/X..	1008	3702 → 3702 Len=966
19623	71.326045	10.56.4.96	239.255.255.250	UDP/X..	1008	3702 → 3702 Len=966
19630	72.130528	10.56.5.117	239.255.255.250	UDP/X..	1008	3702 → 3702 Len=966
19721	72.727982	129.21.180.208	239.255.255.250	UDP/X..	1008	3702 → 3702 Len=966

Output pktsniffer:

```
• (base) aleem@Abduls-MacBook-Air-3 CSIC651 % python pktsniffer.py -r test_wifi.pcap host 10.48.4.102 or port 3702
Total packets read: 28733
All packets were read from the pcap file.

Filter applied: host 10.48.4.102 or port 3702
Packets matching filter: 79

Ethernet Header Information:
 packet_size_bytes destination_mac_address source_mac_address ether_type
0 1008 26:80:df:ad:5f:7a 30:b6:4f:86:fe:2d IPv4
1 1008 26:80:df:ad:5f:7a 30:b6:4f:86:fe:2d IPv4
2 1008 26:80:df:ad:5f:7a 30:b6:4f:86:fe:2d IPv4
3 1008 26:80:df:ad:5f:7a 30:b6:4f:86:fe:2d IPv4
4 1008 26:80:df:ad:5f:7a 30:b6:4f:86:fe:2d IPv4
.. ...
74 1008 26:80:df:ad:5f:7a 30:b6:4f:86:fe:2d IPv4
75 1008 26:80:df:ad:5f:7a 30:b6:4f:86:fe:2d IPv4
76 1008 26:80:df:ad:5f:7a 30:b6:4f:86:fe:2d IPv4
77 802 26:80:df:ad:5f:7a 30:b6:4f:86:fe:2d IPv4
78 802 26:80:df:ad:5f:7a 30:b6:4f:86:fe:2d IPv4

[79 rows x 4 columns]

IP Header Information:
 version header_length_bytes Type of Service total_length_bytes identification flags fragment_offset time_to_live protocol header_checksum source_ip_address destination_ip_address
0 4 20 0 994 51386 DF 0 3 udp 44304 10.48.4.102 239.255.255.250
1 4 20 0 994 51386 DF 0 3 udp 44304 10.48.4.102 239.255.255.250
2 4 20 0 994 11877 DF 0 3 udp 17637 10.56.6.143 239.255.255.250
3 4 20 0 994 64135 DF 0 3 udp 26638 129.21.160.101 239.255.255.250
4 4 20 0 994 64135 DF 0 4 udp 26382 129.21.160.101 239.255.255.250
.. ...
74 4 20 0 994 25193 DF 0 3 udp 3294 10.56.10.146 239.255.255.250
75 4 20 0 994 43815 DF 0 3 udp 58830 10.16.233.10 239.255.255.250
76 4 20 0 994 19860 DF 0 3 udp 9344 10.56.7.197 239.255.255.250
77 4 20 0 788 53564 DF 0 9 udp 41125 10.9.2.244 239.255.255.250
78 4 20 0 788 51922 DF 0 8 udp 17762 10.56.101.114 239.255.255.250

[79 rows x 12 columns]

Encapsulated Packet Header Information (TCP / UDP / ICMP / ICMPv6):
 protocol source_port destination_port sequence_number acknowledgment_number header_length_bytes flags window_size checksum urgent_pointer length_bytes icmp_type icmp_code
0 udp 3702 3702 <NA> <NA> 8 None <NA> 9759 <NA> 974 <NA> <NA>
1 udp 3702 3702 <NA> <NA> 8 None <NA> 9759 <NA> 974 <NA> <NA>
2 udp 3702 3702 <NA> <NA> 8 None <NA> 31196 <NA> 974 <NA> <NA>
3 udp 3702 3702 <NA> <NA> 8 None <NA> 15950 <NA> 974 <NA> <NA>
4 udp 3702 3702 <NA> <NA> 8 None <NA> 15950 <NA> 974 <NA> <NA>
.. ...
74 udp 3702 3702 <NA> <NA> 8 None <NA> 5733 <NA> 974 <NA> <NA>
75 udp 3702 3702 <NA> <NA> 8 None <NA> 52591 <NA> 974 <NA> <NA>
76 udp 3702 3702 <NA> <NA> 8 None <NA> 62628 <NA> 974 <NA> <NA>
77 udp 55652 3702 <NA> <NA> 8 None <NA> 36626 <NA> 768 <NA> <NA>
78 udp 35716 3702 <NA> <NA> 8 None <NA> 29135 <NA> 768 <NA> <NA>
```

We can see that the pktsniffer successfully locates the host or all the packets on the specific port.

4. Wireshark actually translates!

An interesting observation from comparing the two tools is how much protocol interpretation Wireshark performs that pktsniffer (based on scapy) does not attempt. In the capture, traffic on port 443 between 2620:149:a33:f000:... and 2620:8d:8000:e018:... is labeled by Wireshark's Protocol column as TLSv1.x rather than just TCP, and the Info column further breaks individual segments down into application-layer detail such as "Continuation Data" (a TLS record that spans multiple TCP segments) and "Out-Of-Order" / "TCP Retransmission" flags on specific frames.

19181	53.493848	2620:149:a33:f000::...	2620:8d:8000:e018::...	TCP	1294	[TCP Retransmission]	443 → 58737 [ACK] Seq=6028837 Ack=33996 Win=32256 Len=1208 TSval=1861475
19182	53.493848	2620:149:a33:f000::...	2620:8d:8000:e018::...	TCP	1294	[TCP Retransmission]	443 → 58737 [ACK] Seq=6030045 Ack=33996 Win=32256 Len=1208 TSval=1861475
19183	53.493849	2620:149:a33:f000::...	2620:8d:8000:e018::...	TCP	1294	[TCP Retransmission]	443 → 58737 [ACK] Seq=6031253 Ack=33996 Win=32256 Len=1208 TSval=1861475
19184	53.493850	2620:149:a33:f000::...	2620:8d:8000:e018::...	TCP	1294	[TCP Retransmission]	443 → 58737 [ACK] Seq=6032461 Ack=33996 Win=32256 Len=1208 TSval=1861475
19185	53.493850	2620:149:a33:f000::...	2620:8d:8000:e018::...	TCP	1294	[TCP Retransmission]	443 → 58737 [ACK] Seq=6033669 Ack=33996 Win=32256 Len=1208 TSval=1861475
19186	53.493851	2620:149:a33:f000::...	2620:8d:8000:e018::...	TCP	1294	[TCP Retransmission]	443 → 58737 [ACK] Seq=6034877 Ack=33996 Win=32256 Len=1208 TSval=1861475
19187	53.493851	2620:149:a33:f000::...	2620:8d:8000:e018::...	TCP	1294	[TCP Retransmission]	443 → 58737 [ACK] Seq=6036085 Ack=33996 Win=32256 Len=1208 TSval=1861475
19188	53.493852	2620:149:a33:f000::...	2620:8d:8000:e018::...	TCP	1294	[TCP Retransmission]	443 → 58737 [ACK] Seq=6037293 Ack=33996 Win=32256 Len=1208 TSval=1861475
19189	53.493852	2620:149:a33:f000::...	2620:8d:8000:e018::...	TCP	1294	[TCP Retransmission]	443 → 58737 [ACK] Seq=6038501 Ack=33996 Win=32256 Len=1208 TSval=1861475
19190	53.493853	2620:149:a33:f000::...	2620:8d:8000:e018::...	TCP	1294	[TCP Retransmission]	443 → 58737 [ACK] Seq=6039709 Ack=33996 Win=32256 Len=1208 TSval=1861475
19191	53.493854	2620:149:a33:f000::...	2620:8d:8000:e018::...	TCP	1294	[TCP Retransmission]	443 → 58737 [ACK] Seq=6040917 Ack=33996 Win=32256 Len=1208 TSval=1861475
19192	53.493854	2620:149:a33:f000::...	2620:8d:8000:e018::...	TCP	1294	[TCP Retransmission]	443 → 58737 [ACK] Seq=6042125 Ack=33996 Win=32256 Len=1208 TSval=1861475
19193	53.493854	2620:149:a33:f000::...	2620:8d:8000:e018::...	TLSv1..	1294	[TCP Out-Of-Order]	, Continuation Data
19194	53.493855	2620:149:a33:f000::...	2620:8d:8000:e018::...	TLSv1..	1294	[TCP Out-Of-Order]	, Continuation Data
19195	53.493856	2620:149:a33:f000::...	2620:8d:8000:e018::...	TLSv1..	1294	[TCP Out-Of-Order]	, Continuation Data
19196	53.493856	2620:149:a33:f000::...	2620:8d:8000:e018::...	TLSv1..	1294	[TCP Out-Of-Order]	, Continuation Data
19197	53.493857	2620:149:a33:f000::...	2620:8d:8000:e018::...	TLSv1..	1294	[TCP Out-Of-Order]	, Continuation Data
19198	53.493857	2620:149:a33:f000::...	2620:8d:8000:e018::...	TLSv1..	1294	[TCP Out-Of-Order]	, Continuation Data
19199	53.493858	2620:149:a33:f000::...	2620:8d:8000:e018::...	TLSv1..	1294	[TCP Out-Of-Order]	, Continuation Data
19200	53.493858	2620:149:a33:f000::...	2620:8d:8000:e018::...	TLSv1..	1294	[TCP Out-Of-Order]	, Continuation Data
19201	53.493859	2620:149:a33:f000::...	2620:8d:8000:e018::...	TLSv1..	1294	[TCP Out-Of-Order]	, Continuation Data
19202	53.493859	2620:149:a33:f000::...	2620:8d:8000:e018::...	TLSv1..	1294	[TCP Out-Of-Order]	, Continuation Data
19203	53.493860	2620:149:a33:f000::...	2620:8d:8000:e018::...	TLSv1..	1294	[TCP Out-Of-Order]	, Continuation Data
19204	53.493860	2620:149:a33:f000::...	2620:8d:8000:e018::...	TLSv1..	1294	[TCP Out-Of-Order]	, Continuation Data
19205	53.493861	2620:149:a33:f000::...	2620:8d:8000:e018::...	TCP	1294	[TCP Out-Of-Order]	443 → 58737 [ACK] Seq=6057829 Ack=33996 Win=32256 Len=1208 TSval=186147557
19206	53.493861	2620:149:a33:f000::...	2620:8d:8000:e018::...	TLSv1..	1294	[TCP Out-Of-Order]	, Continuation Data
19207	53.493862	2620:149:a33:f000::...	2620:8d:8000:e018::...	TLSv1..	1294	[TCP Out-Of-Order]	, Continuation Data
19208	53.493862	2620:149:a33:f000::...	2620:8d:8000:e018::...	TLSv1..	1294	[TCP Out-Of-Order]	, Continuation Data
19209	53.493863	2620:149:a33:f000::...	2620:8d:8000:e018::...	TLSv1..	1294	[TCP Out-Of-Order]	, Continuation Data
19210	53.493864	2620:149:a33:f000::...	2620:8d:8000:e018::...	TLSv1..	1294	[TCP Out-Of-Order]	, Continuation Data
19211	53.493864	2620:149:a33:f000::...	2620:8d:8000:e018::...	TLSv1..	1294	[TCP Out-Of-Order]	, Continuation Data
19212	53.493895	2620:149:a33:f000::...	2620:8d:8000:e018::...	TLSv1..	1294	[TCP Out-Of-Order]	, Continuation Data
19213	53.493896	2620:149:a33:f000::...	2620:8d:8000:e018::...	TLSv1..	1294	[TCP Out-Of-Order]	, Continuation Data
19214	53.493896	2620:149:a33:f000::...	2620:8d:8000:e018::...	TLSv1..	1294	[TCP Out-Of-Order]	, Continuation Data
19215	53.493897	2620:149:a33:f000::...	2620:8d:8000:e018::...	TLSv1..	1294	[TCP Out-Of-Order]	, Continuation Data

A Snippet from wireshark representing TCP connection experienced apparent packet loss, retransmission, and packet reordering during the capture.

4.1 What Wireshark does better?

- **Upper-layer dissection:** Wireshark reassembles the raw TCP byte stream and hands it to a chain of protocol dissectors — for port 443, it recognizes the TLS record format (handshake, application data, etc.) and, for QUIC (typically UDP/443), it recognizes the QUIC packet header format directly on top of UDP. It relabels the Protocol column accordingly (TLSv1.2, TLSv1.3, QUIC) instead of leaving it as generic TCP/UDP. pktsniffer only implements the IP/TCP/UDP/ICMP header layer, so it has no equivalent dissector and reports every one of these packets simply as tcp.
- **Stream reassembly:** Labels like "Continuation Data" only make sense because Wireshark tracks the entire TCP stream's sequence-number space and reassembles segments in byte order to reconstruct the TLS record boundaries. Pktsniffer parses each packet in isolation, so it can report a segment's own sequence/ack numbers but has no concept of "packet retransmission or continuity."

5. Conclusion

The pktsniffer (based on scapy) shows you the raw header fields packet-by-packet, and provides a summary of the required headers and filters based on clauses such as ip, tcp, udp, host, net, port, and icmp.